

cis112 Tree

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-05-03

Content

- 1 Motivation
- 2 Definitions
- 3 Tree as a Data Structure
- 4 Balanced Trees
- 5 Implementation

[[1], [2], [3], [4], [5], [6], [7]]

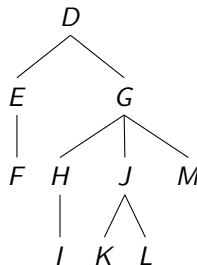


Motivation

Hierarchy

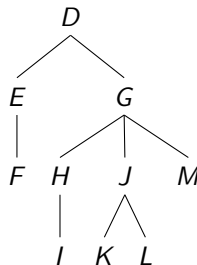
Directory hierarchy

```
MyDisk (D)
|- Private (E)
|  |- Family (F)
|  |- School (G)
|     |- CIS111 (H)
|        |- Project-1 (I)
|        |- CIS112 (J)
|           |- Project-1 (K)
|           |- Project-2 (L)
|     |- CIS114 (M)
```

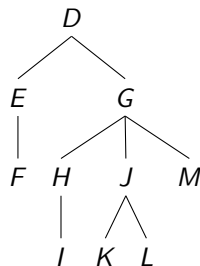
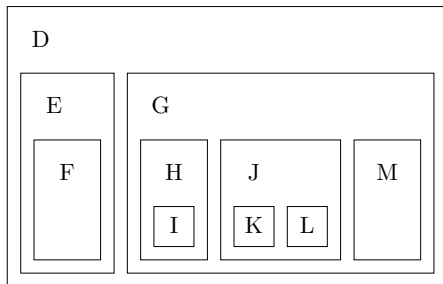


Organizational hierarchy

```
General Manager {  
  VP {  
    Production {  
    }  
  }  
  VP {  
    Human Resources {  
      class I {  
      }  
    }  
    Finance {  
      Accounting {}  
      Treasury {}  
    }  
    Sales {}  
  }  
}
```



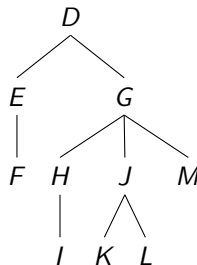
Subset hierarchy



[[1], [3]]

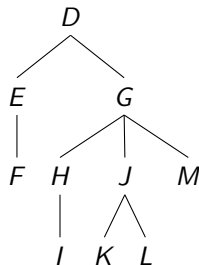
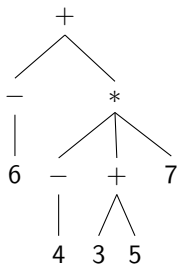
Inner class hierarchy

```
class D {  
    class E {  
        class F {  
        }  
    }  
    class G {  
        class H {  
            class I {  
            }  
        }  
        class J {  
            class K {}  
            class L {}  
        }  
        class M {}  
    }  
}
```



Expressions

$$(-F) + ((-I) \times (K + L) \times M)$$



$$(-6) + (-4) \times (3 + 5) \times 7$$

Linguistics

“the cat sat on the mat.”

S: Sentence

N: Noun

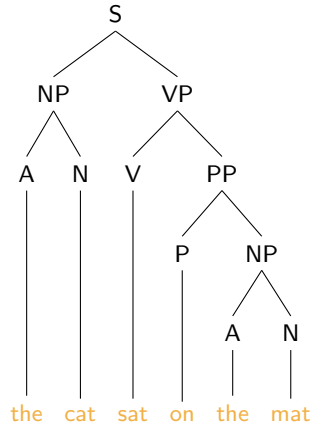
V: Verb

A: Article

NP: Noun Phrase

PP: Prepositional Phrase

VP: Verb Phrase



Linguistics

“the girl hit the ball with a bat.”

S: Sentence

N: Noun

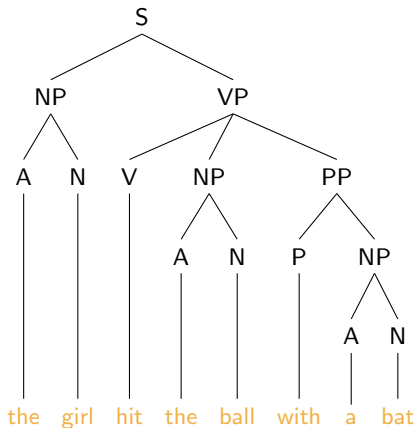
V: Verb

A: Article

NP: Noun Phrase

PP: Prepositional Phrase

VP: Verb Phrase



Definitions

[[\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#), [\[5\]](#), [\[6\]](#), [\[7\]](#)]

Tree and Subtree

Definition (Mathematical)

A **tree** T is a finite, non-empty set of **nodes**

$$T = \{r\} \cup T_1 \cup T_2 \cup \dots \cup T_n,$$

with the following properties:

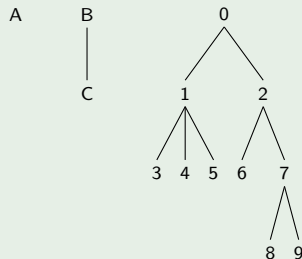
- 1 A designated node of the set, r , is called the **root** of the tree; and
- 2 The remaining nodes are partitioned into $n \geq 0$ subsets, $T_1, T_2, \dots, T_n$, each of which is a tree. T_i is called a **subtree**.

Q. What is the minimum number of vertices in a tree?

Notation.

- $T = \{r, T_1, T_2, \dots, T_n\}$ denotes the tree T .
- V is the set of **vertices** (nodes) in T .
- $v \in V$ is a **vertex** (node) of T .

Example



A is a tree with 1 node.

B is a tree with 2 nodes.

0 is a tree with 9 nodes.

1 is a tree with 4 nodes.

2 is a tree with 5 nodes.

4 is a tree with 1 node.

Degree and Leaf

Definition

Let $T = \{r, T_1, T_2, \dots, T_n\}$ be a tree.
 The **degree** of a node is the number of subtrees associated with that node.

Example

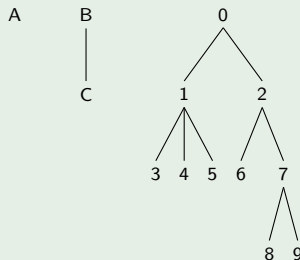
The degree of r in tree $T = \{r, T_1, T_2, \dots, T_n\}$ is n .

Definition

A node of degree 0 is called a **leaf**.

Q. Is it possible to have a negative degree?

Example



Nodes A, C, 3 are leaves.
 Degree of node B is 1.
 Degree of node 0 is 2.
 Degree of node 1 is 3.

Parent, Child and Siblings

Definition

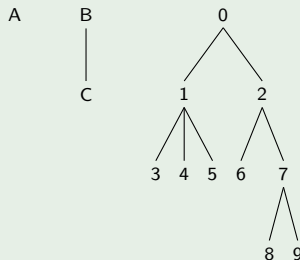
Let $T = \{r, T_1, T_2, \dots, T_n\}$, $n \geq 0$ be a tree T .
 Let r_i be the root of subtree T_i . Then

- r is called the **parent** of r_i .
- r_i is called a **child** of r
- Roots r_i and r_j of distinct subtrees T_i and T_j of tree T are called **siblings**.

Q. What is the minimum and maximum number of

- parents of a node a ?
- children of a node a ?
- siblings of a node a ?

Example



Node B is parent of node C .
 Node C is a child of node B .
 Nodes 3 and 4 are siblings.

Path and Path Length

Definition

Let $T = \{r, T_1, T_2, \dots, T_n\}$ be a tree T . Let V be the set of nodes in T .

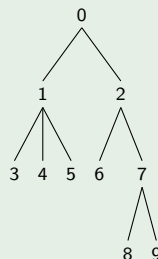
- A **path** is a non-empty sequence of nodes $P = (v_1, v_2, \dots, v_k)$ where, $v_i \in V$ for $1 \leq i \leq k$, such that v_i is parent of v_{i+1} .
- The **length** of path P is $k - 1$.

Remark.

- Direction of a path is from root to leaf.
- There is a **unique** path from root to any node in the tree.

Q. Is (v) a path?

Example



$(2, 7, 9)$ is a path.

$(0, 1)$ is a path.

Unique path to node 8 is the path $(0, 2, 7, 8)$

Ancestor and Descendant

Definition

Let T be a tree with the set of nodes V . Suppose there exists a path P from v_i to v_j for $v_i, v_j \in V$, Then

- the vertex v_i is an **ancestor** of v_j ;
- the vertex v_j is a **descendant** of v_i .

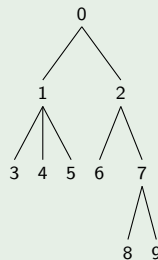
If the length of P is non-zero, then

- the vertex v_i is a **proper ancestor** of v_j ;
- the vertex v_j is a **proper descendant** of v_i .

Remark.

- A path is from ancestor to descendant.
- Vertex v is ancestor of itself.
- Vertex v is descendant of itself.

Example



2 is ancestor of 7.
9 is descendant of 2.

Level (Depth) and Height

Definition

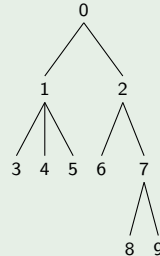
Let T be a tree with the set of nodes V .

- The **level (depth)** of a node $v \in V$ in a tree T is the length of the unique path in T from its root r to the node v .
- The **height** of a node $v \in V$ in a tree T is the length of the longest path from node v to a leaf.
- The **height** of a tree T is the height of its root r .

Remark.

- The root r of T is at level-0.
- The roots of the subtrees of r are at level-1.
- The leaves are at height 0.

Example



Node 0 is at level-0.
Node 1 is at level-1.
Node 4 is at level-2.
Node 9 is at level-3.
Height of node 7 is 1.
Height of node 2 is 2.
Height of node 0 is 3.
Height of **tree** T is 3.

Tree as a Data Structure

N-ary Trees

Definition (Data Structure)

An ***N*-ary tree** T is a finite set of nodes with the following properties:

- 1 Either the set is empty, $T = \emptyset$; or
- 2 The set consists of a **root**, r , and exactly N distinct *N*-ary trees, T_i .
I.e., the remaining nodes are partitioned into $N \geq 0$ subsets, $T_0, T_1, \dots, T_{N-1}$, each of which is an *N*-ary tree such that $T = \{r, T_0, T_1, \dots, T_{N-1}\}$.

Note that

- The **degree** of each node of an *N*-ary tree is either zero or N
- The **empty tree**, $T = \emptyset$, is a tree. That is, it is an object of the same type as a non-empty tree

Binary Trees

Definition (Data Structure)

A **binary tree** T is a finite set of **nodes** with the following properties:

- 1 Either the set is empty, $T = \emptyset$; or
- 2 The set consists of a **root**, r , and exactly two distinct binary trees T_L and T_R , $T = \{r, T_L, T_R\}$.

The tree T_L is called the **left subtree** of T , and the tree T_R is called the **right subtree** of T .

Note that

- The **degree** of each node of an binary tree is either 0, 1 or 2.
- A binary tree of height $h \geq 0$ has at most $2^{h+1} - 1$ nodes
- Therefore the height of a binary tree with n nodes is at least $\lceil \log_2 n + 1 \rceil - 1$.

Ordered Trees

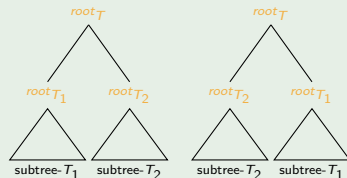
Definition

An **ordered tree** is a tree in which the order of the subtrees matters.

Warning. Unless stated otherwise, all trees we deal with are ordered trees.

Example

Tree $T_{12} = \{r, T_1, T_2\}$ is different than tree $T_{21} = \{r, T_2, T_1\}$.



Balanced Trees

Balanced Trees

Definition

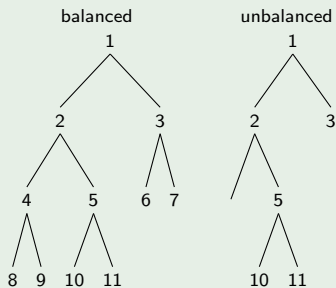
A tree $T = \{r, T_1, T_2, \dots, T_n\}$ is **balanced** iff

- T is empty or
- $T_1, T_2, \dots, T_n$ have “almost the same height”.

Remark.

- Consider “almost the same height” as difference of 1.
- How to **keep a tree balanced** is an important issue that we will deal with

Example



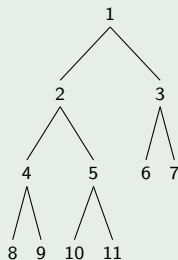
Complete Binary Tree

Definition

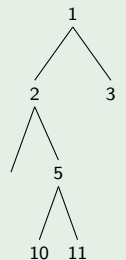
A binary tree $T = \{r, T_L, T_R\}$ is called **complete binary tree** iff each node has either 0 or 2 degrees.

Example

complete binary tree



not



due to node 2.

Full Binary Tree

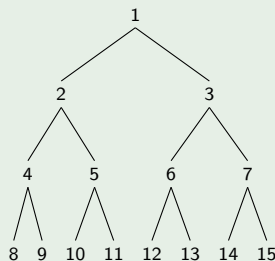
Definition

A binary tree T is called **full binary tree** iff all the leafs in T are at level h . [\[\[7\]\]](#)

Remark.

- Some books, such as [\[\[2\]\]](#), calls it *complete binary tree*.
- A full binary tree is a special form with the property that **maximum possible number of nodes in a minimum possible height**.

Example



Relations in Number of Nodes and Height

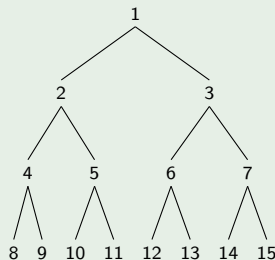
In a full binary tree

- at **level-0**, there is only $2^0 = 1$ node: 1
- at **level-1**, there are $2^1 = 2$ nodes: 2, 3
- at **level-2**, there are $2^2 = 4$ nodes: 4, 5, 6, 7
- at **level-3**, there are $2^3 = 8$ nodes: 8, 9, 10, 11, 12, 13, 14, 15

Generalization.

- In a **full binary tree**
 - at level- ℓ , there are 2^ℓ nodes.
 - height $h \implies n = 2^{h+1} - 1$.
 - number of nodes $n \implies h = \log_2(n + 1) - 1$.
- In any **binary tree**, $n \leq 2^{h+1} - 1$.

Example



A full binary tree. Levels: 0, 1, 2, and 3. The number of nodes: $n = 15$ and height: $h = 3$. Then $n = 2^{h+1} - 1$ and $h = \log_2(n + 1)$.

Theorem.

$$(1 + x^1 + \dots + x^n)(x - 1) = x^{n+1} - 1$$

Relations in Number of Nodes and Height

Height of a tree is very important

- In **search**, number of steps is **directly related to the height**
- The relation between n and h is **logarithmic**, i.e.,
 $h = \lceil \log_2(n + 1) - 1 \rceil$. It is due to nonlinearity of tree data structure
- Therefore, searching in trees is **$O(\log n)$** rather than **$O(n)$** , which is a big improvement for big n values.
- Because of these nice properties, trees are very frequently used in CS.

In any **binary tree**,

- Given h , $n \leq 2^{h+1} - 1$
- Given n , $h \geq \lceil \log_2(n + 1) - 1 \rceil$

n	$\log_2(n + 1) - 1$	h
1,000	8.97	9
1,000,000	18.93	19
10,000,000	22.25	23
100,000,000	25.58	26
1,000,000,000	28.90	29

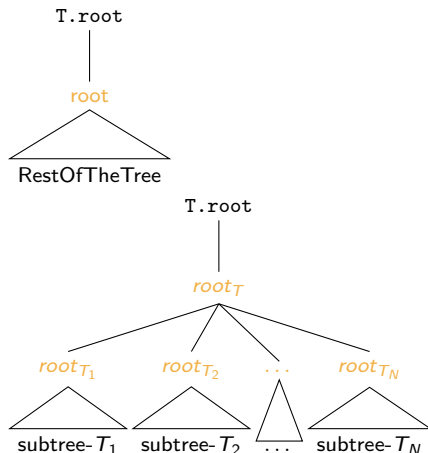
Implementation

Tree

Tree T is a data structure

- with a $T.root$ pointing to the $root$ node of the tree
- $T.root.parent = NIL$
- Let x be a node of T
 - If $x.parent = NIL$, then x is the $root$, i.e., $root$ is the only node with parent is NIL
 - If $x.subtree_i = NIL$, then x has no subtree T_i i.e., the subtree T_i is empty

[[1], [2], [3], [4], [5], [6], [7]]



N-ary Tree

Algorithm 1: Node for N -ary tree:

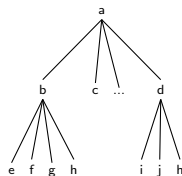
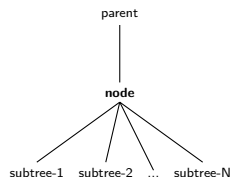
General case

```

1 begin
2   data
3   parent           // reference to the parent
4   child1           // reference to the first subtree
5   child2           // reference to the second subtree
6   ...
7   childN           // reference to the  $N$ 'th subtree
    
```

Remark. In the most general case of a tree, each node may have different number N of subtrees.

This is a problem.



N-ary Trees: Left-child, right-sibling representation

Algorithm 2: Node for Left-child, right-sibling

```

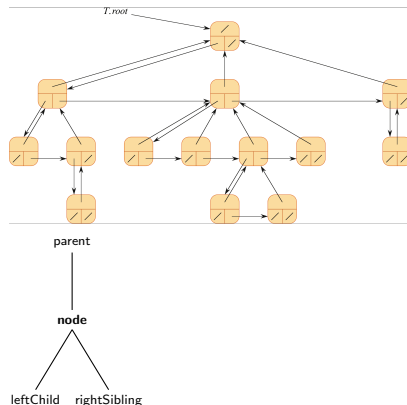
1 begin
2   data
3   parent           // reference to the parent
4   leftChild        // reference to the left child
5   rightSibling     // reference to the right sibling

```

Use node with three pointers:

- **parent**: points the parent
- **leftChild**: points the left child
- **rightSibling**: points the right sibling

Remark. Now node becomes uniform, i.e., independent of degree.



Binary Tree

Algorithm 3: Node for binary tree

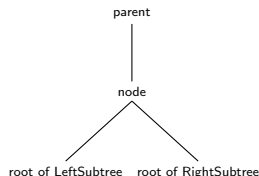
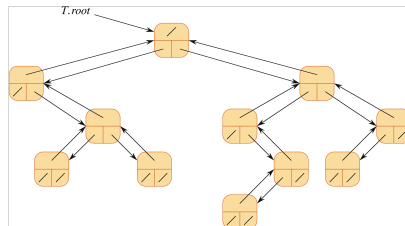
```

1 begin
2   data
3   parent           // reference to the parent
4   left             // reference to the left child
5   right            // reference to the right child
    
```

In **binary tree**, each node has 0, 1 or 2 subtrees.

Use node with three pointers:

- **parent**: points the parent
- **left**: points the left child
- **right**: points the right child



MyNode in Java

Algorithm 4: Node for binary tree

```

1 begin
2   data
3   parent           // reference to the parent
4   left             // reference to the left child
5   right            // reference to the right child
    
```

MyNode implements interface **NodeBinaryI**.

```

1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
    
```

```

1 public class MyNode<T> implements NodeBinaryI<T>{
2
3     public T data;
4     public MyNode<T> left;
5     public MyNode<T> right;
6
7     public MyNode() {
8         this(null);
9     }
10
11    public MyNode(T data) {
12        this.data = data;
13        left = null;
14        right = null;
15    }
16
17    ...
18
19    @Override
20    public String toString() {
21        return "[MyNode: data=" + data + "]";
22    }
23
24 }
    
```

MyTree in Java: Constructors

```

1 public class MyBinaryTree<T> {
2
3     private MyNode<T> root;
4
5     public MyBinaryTree() {
6         root = null;
7     }
8
9     public MyBinaryTree(T data) {
10         root = new MyNode<>(data);
11     }
12
13     public MyBinaryTree(
14         T data
15         , MyBinaryTree<T> left
16         , MyBinaryTree<T> right
17     ) {
18         root = new MyNode<>(data);
19         if (left == null) {
20             root.left = null;
21         } else {
22             root.left = left.root;
23         }
24         if (right == null) {
25             root.right = null;
26         } else {
27             root.right = right.root;
28         }
29     }
30     ...
31 }
    
```

```

1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
    
```

```

1 public class MyNode<T> implements NodeBinaryI<T>{
2
3     public T data;
4     public MyNode<T> left;
5     public MyNode<T> right;
6
7     public MyNode() {
8         this(null);
9     }
10
11     public MyNode(T data) {
12         this.data = data;
13         left = null;
14         right = null;
15     }
16
17     ...
18
19     @Override
20     public String toString() {
21         return "[MyNode: data=" + data + "]";
22     }
23
24 }
    
```

Implementation

Navigation in Trees

Navigation

```

1 public class MyBinaryTree<T> {
2
3     private MyNode<T> root;
4     ...
5     public String traverseInOrder() {
6         return LibTree.traverseInOrder(root);
7     }
8
9     public String traversePreOrder() {
10        return LibTree.traversePreOrder(root);
11    }
12
13    public String traversePostOrder() {
14        return LibTree.traversePostOrder(root);
15    }
16    ...
17 }
```

```

1 public class LibTree<T> {
2     public static <T> String traverseInOrder(
3         NodeBinaryInterface<T> node
4     ) {
5         if (node == null) {
6             return "";
7         }
8         String s = "";
9         s += traverseInOrder(node.left());
10        s += node.toString();
11        s += traverseInOrder(node.right());
12        return s;
13    }
14 }
```

```

1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
```

```

1 public class MyNode<T> implements NodeBinaryI<T>{
2
3     public T data;
4     public MyNode<T> left;
5     public MyNode<T> right;
6
7     @Override
8     public NodeBinary left() {
9         return left;
10    }
11
12    @Override
13    public NodeBinary right() {
14        return right;
15    }
16
17    @Override
18    public T data() {
19        return data;
20    }
21
22 }
```

Navigation: InOrder

```

1 public class MyBinaryTree<T> {
2
3     private MyNode<T> root;
4     ...
5     public String traverseInOrder() {
6         return LibTree.traverseInOrder(root);
7     }
8
9     public String traversePreOrder() {
10        return LibTree.traversePreOrder(root);
11    }
12
13    public String traversePostOrder() {
14        return LibTree.traversePostOrder(root);
15    }
16    ...
17 }

```

```

1 public class LibTree<T> {
2     public static <T> String traverseInOrder(
3         NodeBinaryInterface<T> node
4     ) {
5         if (node == null) {
6             return "";
7         }
8         String s = "";
9         s += traverseInOrder(node.left());
10        s += node.toString();
11        s += traverseInOrder(node.right());
12        return s;
13    }
14 }

```



```

1 public class MyBinaryTreeConstructor {
2     public static MyBinaryTree<String>
3         constructBT_S_Expression() {
4         // ( 8 + 7 ) * ( 5 - 2 )
5         MyBinaryTree<String> tree;
6         tree = //
7             new MyBinaryTree<String>("x", //
8                 new MyBinaryTree<String>("+", //
9                     new MyBinaryTree<String>("8"), //
10                    new MyBinaryTree<String>("7") //
11                ), //
12                new MyBinaryTree<String>("-", //
13                    new MyBinaryTree<String>("5"), //
14                    new MyBinaryTree<String>("2") //
15                ) //
16            );
17        // check
18        tree.plot();
19        System.out.println("\ncanonical:" + tree.canonical());
20        System.out.println("\ntraverseInOrder:" + tree.traverseInOrder());
21        return tree;
22    }
23 }

```

Navigation: InOrder

```
1 public class MyBinaryTree<T> {
2
3     private MyNode<T> root;
4     ...
5     public String traverseInOrder() {
6         return LibTree.traverseInOrder(root);
7     }
8
9     public String traversePreOrder() {
10        return LibTree.traversePreOrder(root);
11    }
12
13    public String traversePostOrder() {
14        return LibTree.traversePostOrder(root);
15    }
16    ...
17 }
```

```
1 public class LibTree<T> {
2     public static <T> String traverseInOrder(
3         NodeBinaryInterface<T> node
4     ) {
5         if (node == null) {
6             return "";
7         }
8         String s = "";
9         s += traverseInOrder(node.left());
10        s += node.toString();
11        s += traverseInOrder(node.right());
12        return s;
13    }
14 }
```



```
1 public class
2     public sta
3     constructBT_S_Expression() {
4         // ( 8 + 7 ) * ( 5 - 2 )
5         MyBinaryTree<String> tree;
6         tree = //
7             new MyBinaryTree<String>("x", //
8                 new MyBinaryTree<String>("+", //
9                     new MyBinaryTree<String>("8"), //
10                    new MyBinaryTree<String>("7") //
11                ), //
12                new MyBinaryTree<String>("-", //
13                    new MyBinaryTree<String>("5"), //
14                    new MyBinaryTree<String>("2") //
15                ) //
16            );
17        // check
18        tree.plot();
19        System.out.println("\ncanonical:" + tree.
20            canonical());
21        System.out.println("\ntraverseInOrder:" + tree
22            .traverseInOrder());
23        return tree;
24    }
```

```

/2
/-
\5
3
4 >*
5 /7
6 /+
7 \8
8
9 canonical:[8]\[+]\[7]\[*]\[5]\[-]\[2]\
10
11 traverseInOrder:
12 [MyNode: data=8]
13 [MyNode: data=+]
14 [MyNode: data=7]
15 [MyNode: data=+]
16 [MyNode: data=5]
17 [MyNode: data=-]
18 [MyNode: data=2]

```

Navigation: InOrder, PreOrder, PostOrder

```

1 public class MyBinaryTree<T> {
2
3     private MyNode<T> root;
4     ...
5     public String traverseInOrder() {
6         return LibTree.traverseInOrder(root);
7     }
8
9     public String traversePreOrder() {
10        return LibTree.traversePreOrder(root);
11    }
12
13    public String traversePostOrder() {
14        return LibTree.traversePostOrder(root);
15    }
16    ...
17 }

```

```

1 public class LibTree<T> {
2     public static <T> String traverseInOrder(
3         NodeBinaryInterface<T> node
4     ) {
5         if (node == null) {
6             return "";
7         }
8         String s = "";
9         s += traverseInOrder(node.left());
10        s += node.toString();
11        s += traverseInOrder(node.right());
12        return s;
13    }
14 }

```

```

1 public class LibTree<T> {
2     ...
3     public static <T> String traversePreOrder(
4         NodeBinaryInterface<T> node) {
5         if (node == null) {
6             return "";
7         }
8         String s = "";
9         s += node.toString();
10        s += traversePreOrder(node.left());
11        s += traversePreOrder(node.right());
12        return s;
13    }
14
15    public static <T> String traversePostOrder(
16        NodeBinaryInterface<T> node) {
17        if (node == null) {
18            return "";
19        }
20        String s = "";
21        s += traversePostOrder(node.left());
22        s += traversePostOrder(node.right());
23        s += node.toString();
24        return s;
25    }
26 }

```

Implementation

Other Recursive Methods

Plot

```
1 public class MyBinaryTree<T> {
2     private MyNode<T> root;
3     ...
4     public void plot() {
5         LibTree.plot(root);
6         System.out.println();
7     }
8 }
```

```
1 public class LibTree<T> {
2     public static void plot(NodeBinaryI<T> node) {
3         plot(node, 0, ">");
4     }
5
6     private static void plot(NodeBinaryI<T> node,
7         int level, String leftRight) {
8         // right subtree
9         if (node.right() != null) {
10             plot(node.right(), level + 1, "/");
11         }
12         // print the node
13         String indent = " ".repeat(level);
14         System.out.println(indent + leftRight + node
15             .data());
16         // left subtree
17         if (node.left() != null) {
18             plot(node.left(), level + 1, "\\");
19         }
20     }
21 }
```

```
1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
```

Example



Plot

```
1 public class MyBinaryTree<T> {
2     private MyNode<T> root;
3     ...
4     public void plot() {
5         LibTree.plot(root);
6         System.out.println();
7     }
8 }
```

```
1 public class LibTree<T> {
2     public static void plot(NodeBinaryI<T> node) {
3         plot(node, 0, ">");
4     }
5
6     private static void plot(NodeBinaryI<T> node,
7         int level, String leftRight) {
8         // right subtree
9         if (node.right() != null) {
10             plot(node.right(), level + 1, "/");
11         }
12         // print the node
13         String indent = " ".repeat(level);
14         System.out.println(indent + leftRight + node
15             .data());
16         // left subtree
17         if (node.left() != null) {
18             plot(node.left(), level + 1, "\\");
19         }
20     }
21 }
```

```
1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
```

Example



```
1 /3
2 \6
3 >1
4 /5
5 \2
6 \4
```

Height

```
1 public class MyBinaryTree<T> {
2     private MyNode<T> root;
3     ...
4     public int height() {
5         return LibTree.height(root);
6     }
7 }
```

```
1 public class LibTree<T> {
2     ...
3     public static <T> int height(
4         NodeBinaryInterface<T> node) {
5         if (node == null) {
6             return 0;
7         } else if (node.left() == null//
8             && node.right() == null) {
9             return 0;
10        } else {
11            int h = 1 + Math.max(//
12                height(node.left())//
13                , height(node.right())//
14            );
15            return h;
16        }
17        ...
18 }
```

```
1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
```

Example



Call `MyBinaryTree.Test.height_constructBT_S.1.6()`
`MyBinaryTree_Test.height_constructBT_S.1.6()`

Height

```
1 public class MyBinaryTree<T> {
2     private MyNode<T> root;
3     ...
4     public int height() {
5         return LibTree.height(root);
6     }
7 }
```

```
1 public interface NodeBinaryI<T> {
2     NodeBinary<T> left();
3     NodeBinary<T> right();
4     T data();
5 }
```

```
1 public class LibTree<T> {
2     ...
3     public static<T> int height(
4         NodeBinaryInterface<T> node) {
5         if (node == null) {
6             return 0;
7         } else if (node.left() == null//
8             && node.right() == null) {
9             return 0;
10        } else {
11            int h = 1 + Math.max(//
12                height(node.left()),//
13                height(node.right())//
14            );
15            return h;
16        }
17    }
18 }
```

Example



Call `MyBinaryTree.Test.height_constructBT_S_1_6()`
`MyBinaryTree_Test.height_constructBT_S_1_6()`

```
1 -constructBT_S_1_6
2 /3
3 \6
4 >1
5 /5
6 \2
7 \4
8
9 node:1, height:2
10 node:2, height:1
11 node:3, height:1
12 node:4, height:0
13 node:5, height:0
14 node:6, height:0
```

References I

- [1] D. E. Knuth, *The Art of Computer Programming: Fundamental Algorithms, volume 1*, 2nd ed. Addison-Wesley Professional, 1973.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms (CLRS)*, 4th ed. MIT Press, 2022.
- [3] B. Preiss, *Data Structures and Algorithms with Object-Oriented Design Patterns in Java*. Wiley, 1999.
- [4] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*. Addison Wesley, 1974.
- [5] E. Horowitz and S. Sahni, *Fundamentals of Data Structures*. Pitman, 1982.
- [6] C. S. Horstmann, *Big Java: Early Objects*, 7th ed. John Wiley & Sons, 2019.
- [7] R. P. Grimaldi, *Discrete and Combinatorial Mathematics*, 5th ed. Pearson Education India, 2006.